

Array Functions and Methods

Name	Description
<code>max(array)</code>	Returns the maximum value of an array
<code>min(array)</code>	Returns the minimum value of an array
<code>sum(array)</code>	Returns the sum of the values in an array
<code>abs(num)</code> , <code>np.abs(array)</code>	Takes the absolute value of a number or each number in an array
<code>round(num)</code> , <code>np.round(array)</code> <code>round(num, decimals)</code> , <code>np.round(array, decimals)</code>	Rounds a number or an array of numbers to the nearest integer. If <code>decimals</code> (integer) is included, rounds to that many digits. If <code>decimals</code> is negative, remove that many digits of precision.
<code>len(array)</code>	Returns the length (number of elements) of an array
<code>make_array(val1, val2, ...)</code>	Makes a numpy array with the values passed in
<code>np.average(array)</code> <code>np.mean(array)</code>	Returns the mean value of an array
<code>np.diff(array)</code>	Returns a new array of size <code>len(arr) - 1</code> with elements equal to the difference between adjacent elements; <code>val_2 - val_1</code> , <code>val_3 - val_2</code> , etc.
<code>np.arange(start, stop, step)</code> <code>np.arange(start, stop)</code> <code>np.arange(stop)</code>	An array of numbers starting with <code>start</code> , going up in increments of <code>step</code> , and going up to but excluding <code>stop</code> . When <code>start</code> and/or <code>step</code> are left out, default values are used in their place. Default <code>step</code> is 1; default <code>start</code> is 0.
<code>array.item(index)</code>	Returns the i-th item in an array (remember Python indices start at 0!)
<code>np.random.choice(array, n)</code> <code>np.random.choice(array)</code>	Picks one (by default) or some number (<code>n</code>) of items from <code>array</code> at random with replacement.
<code>np.count_nonzero(array)</code>	Returns the number of non-zero (or <code>True</code>) elements in an array.
<code>np.append(array, item)</code>	Returns a copy of the input array with <code>item</code> appended to the end.

Table Methods

Name	Description	Input	Output
<code>Table()</code>	Create an empty table, usually to extend with data	None	An empty Table
<code>Table().read_table(filename)</code>	Create a table from a data file	string: the name of the file	Table with the contents of the data file
<code>tbl.with_columns(name, values)</code> <code>tbl.with_columns(n1, v1, n2, v2, ...)</code>	A table with an additional or replaced column or columns. <code>name</code> is a string for the name of a column, <code>values</code> is an array	1. string: the name of the new column; 2. array: the values in that column	Table: a copy of the original Table with the new columns added
<code>tbl.column(column_name)</code>	The values of a column (an array)	string or int: the column name or index	array: the values in that column
<code>tbl.num_rows</code> <code>(tbl.num_columns)</code>	Compute the number of rows(columns) in a table	None	int: the number of rows (columns) in the table
<code>tbl.select(col1, col2, ...)</code>	Create a copy of a table with only some of the columns. Each column is the column name or index.	string or int: column name(s) or index(es)	Table with the selected columns
<code>tbl.drop(col1, col2, ...)</code>	Create a copy of a table without some of the columns. Each column is the column name or index.	string or int: column name(s) or index(es)	Table without the selected columns
<code>tbl.sort(column_name_or_index)</code>	Create a copy of a table sorted by the values in a column. Defaults to ascending order unless <code>descending = True</code> is included.	1. string or int: column index or name 2. (Optional) <code>descending=True</code>	Table: a copy of the original with the column sorted

<code>tbl.where(column, predicate)</code>	Create a copy of a table with only the rows that match some <i>predicate</i> . See <code>Table.where</code> predicates below.	1. string or int: column name or index 2. <code>are(...)</code> predicate	Table: a copy of the original table with only the rows that match the predicate
<code>tbl.take(row_indices)</code>	A table with only the rows at the given indices. <code>row_indices</code> is either an array of indices or an integer corresponding to one index.	array of ints: the indices of the rows to be included in the Table OR int: the index of the row to be included	Table: a copy of the original table with only the rows at the given indices
<code>tbl.scatter(x_column, y_column)</code>	Draws a scatter plot consisting of one point for each row of the table. Note that <code>x_column</code> and <code>y_column</code> must be strings specifying column names. Include optional argument <code>fit_line=True</code> if you want to draw a line of best fit for each set of points.	1. string: name of the column on the x-axis 2. string: name of the column on the y-axis 3. (Optional) <code>fit_line=True</code>	None: draws a scatter plot
<code>tbl.plot(x_column, y_column)</code> <code>tbl.plot(x_column)</code>	Draw a line graph consisting of one point for each row of the table. If you only specify one column, it will plot the rest of the columns on the y-axis as different colored lines.	1. string: name of the column on the x-axis 2. string: name of the column on the y-axis	None: draws a line graph
<code>tbl.barh(categories)</code> <code>tbl.barh(categories, values)</code>	Displays a bar chart with bars for each category in a column, with height proportional to the corresponding frequency. <code>values</code> argument unnecessary if table has only a column of	1. string: name of the column with categories 2. (Optional) string: the name of the column with values for corresponding categories	None: draws a bar chart

	categories and a column of values.		
<code>tbl.hist(column, unit, bins, group)</code>	Generates a histogram of the numerical values in a column. <code>unit</code> and <code>bins</code> are optional arguments, used to label the axes and group the values into intervals (bins), respectively. Bins have the form <code>[a, b)</code> , where <code>a</code> is included in the bin and <code>b</code> is not.	1. string: name of the column with categories 2. (Optional) string: units of x-axis 3. (Optional) array of ints/floats denoting bin boundaries 4. (Optional) string: name of categorical column to draw separate overlaid histograms for	None: draws a histogram
<code>tbl.apply(function)</code> <code>tbl.apply(function, col1, col2, ...)</code>	Returns an array of values resulting from applying a function to each item in a column.	1. function: function to apply to column 2. (Optional) string: name of the column to apply function to (if you have multiple columns, the respective column's values will be passed as the corresponding argument to the function), and if there is no argument, your function will be applied to every row object in <code>tbl</code>	array: contains an element for each value in the original column after applying the function to it
<code>tbl.group(column_or_columns, func)</code>	Group rows by unique values or combinations of values in a column(s). Multiple columns must be entered in array or list form. Other values aggregated by count (default) or optional argument <code>func</code> .	1. string or array of strings: column(s) on which to group 2. (Optional) function: function to aggregate values in cells (defaults to count)	Table: a new Table

<pre>tbl.pivot(col1, col2, values, collect) tbl.pivot(col1, col2)</pre>	A pivot table where each unique value in <code>col1</code> has its own column and each unique value in <code>col2</code> has its own row. Count or aggregate values from a third column, collect with some function. Default <code>values</code> and <code>collect</code> return counts in cells.	1. string: name of column whose unique values will make up columns of pivot table 2. string: name of column whose unique values will make up rows of pivot table 3. (Optional) string: name of column that describes the values of cell 4. (Optional) function: how the values are collected, e.g. <code>sum</code> or <code>np.mean</code>	Table: a new Table
<pre>tblA.join(colA, tblB, colB) tblA.join(colA, tblB)</pre>	Generate a table with the columns of <code>tblA</code> and <code>tblB</code>, containing rows for all values of a column that appear in both tables. Default <code>colB</code> is <code>colA</code>. <code>colA</code> and <code>colB</code> must be strings specifying column names.	1. string: name of a column in <code>tblA</code> with values to join on 2. Table: other Table 3. (Optional) string: if column names are different between Tables, the name of the shared column in <code>tblB</code>	Table: a new Table
<pre>tbl.sample(n) tbl.sample(n, with_replacement)</pre>	A new table where <code>n</code> rows are randomly sampled from the original table; by default, <code>n=tbl.num_rows</code>. Default is with replacement. For sampling without replacement, use argument <code>with_replacement=False</code>. For a non-uniform sample, provide a third argument <code>weights=distribution</code> where <code>distribution</code> is an array or list containing	1. int: sample size 2. (Optional) <code>with_replacement=True</code>	Table: a new Table with <code>n</code> rows

	the probability of each row.		
<code>tbl.row(row_index)</code>	Accesses the row of a table by taking the index of the row as its argument. Note that rows are in general not arrays, as their elements can be of different types. However, you can use <code>.item</code> to access a particular element of a row using <code>row.item(label)</code> .	int: row index	Row object with the values of the row and labels of the corresponding columns

Table.where Predicates

Any of these predicates can be negated by adding `not_` in front of them, e.g.

`are.not_equal_to(Z)` or `are.not_containing(S)`.

Predicate	Description
<code>are.equal_to(Z)</code>	Equal to <code>Z</code>
<code>are.not_equal_to(Z)</code>	Not equal to <code>Z</code>
<code>are.above(x)</code>	Greater than <code>x</code>
<code>are.above_or_equal_to(x)</code>	Greater than or equal to <code>x</code>
<code>are.below(x)</code>	Less than <code>x</code>
<code>are.below_or_equal_to(x)</code>	Less than or equal to <code>x</code>
<code>are.between(x, y)</code>	Greater than or equal to <code>x</code> and less than <code>y</code>
<code>are.between_or_equal_to(x, y)</code>	Greater than or equal to <code>x</code> , and less than or equal to <code>y</code>
<code>are.strictly_between(x, y)</code>	Greater than <code>x</code> and less than <code>y</code>
<code>are.contained_in(A)</code>	Is a substring of <code>A</code> (if <code>A</code> is a string) or an element of <code>A</code> (if <code>A</code> is a list/array)
<code>are.containing(S)</code>	Contains the string <code>S</code>

Miscellaneous Functions

Name	Description	Input	Output
<code>sample_proportions(sample_size, model_proportions)</code>	<code>sample_size</code> should be an integer, <code>model_proportions</code> an array of probabilities that sum up to 1. The function samples <code>sample_size</code> objects from the distribution specified by <code>model_proportions</code> . It returns an array with the same size as <code>model_proportions</code> . Each item in the array corresponds to the proportion of times it was sampled out of the <code>sample_size</code> times.	1. int: sample size 2. array: an array of proportions that should sum to 1	array: each item corresponds to the proportion of times that corresponding item was sampled from <code>model_proportions</code> in <code>sample_size</code> draws, should sum to 1